__/__/____

To clear the array — arr.clear();

To check it is empty or not — arr.empty();
↳ will return True or False

To find max element — int max = *max_element(arr.begin(), arr.end())

To Rotate Right — ans[(i+k)%n] = arr[i]; } k = no. of rotations

To Rotate Left — ans[(i-k+n)%n] = arr[i]; }

To find permutation — next_permutation(arr.begin(), arr.end());
See 12 pages back

String to char array → string str = "hello";
vector<char> arr(str.begin(), str.end());

__/__/____

Array

```
int arr[5] = {5, 3, 2, 11, 14};
```

create — Create a null Array

```
vector<int> arr;
```

size — To find size of Array → `arr.size();`

sort — To sort the array → `sort(arr.begin(), arr.end());`
`sort(arr, arr+n);`

add — To add an element to an Array
↳ only for null Arrays

```
vector<int> arr;
nums[5] = {5, 11, 13, 4, 2};
arr.push_back(nums[0]);
```

remove → `arr.erase(arr.begin() + i);` i = no. to remove

note → If u want to come out of the fxn after checking the problem use return;

```
for(int i=1; i<n; i++){
    if(arr[i] < arr[i-1]{
        cout << "No" << endl;
        return;
    }
}
cout << "Yes" << endl;
```

Problem to check no. are in ascending order.

Combine two array → `a.insert(a.end(), b.begin(), b.end());`

Permutation → to find [1 2 3] → to find next permutation
→ `next_permutation(arr.begin(), arr.end);`

String

__/__/__

int → string to_string(n);

string → int stoi(s);

to reverse string std::reverse(s.begin(), s.end());

uppercase to lowercase → transform(s.begin(), s.end(), s.begin, :: tolower

lower to upper → transform(s.begin(), s.end(), s.begin, :: toupper);

__/__/____

To find no. of permutations

arr : | 1 | 4 | 6 |

```
int num_permutation = 1;
for(int i=2 ; i<arr.size() ; i++){
    num_permutation = num_permutation * i ;
}
return num_permutation ;
```

To find & all the permutation

arr : | 1 | 4 | 6 |

```
sort(arr.begin(), arr.end());
do{
    for(int i=0 ; i<arr.size(); i++){
        cout << arr[i] << " ";
    } cout << endl;
} while (next_permutation(arr.begin(), arr.end()));
```

__/__/____

Reverse an Array

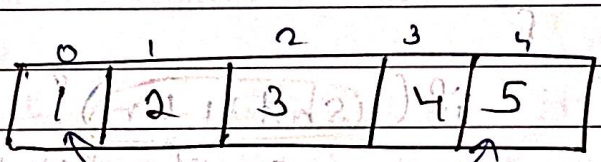
make a new Array & use → newArr[i] = arr[size-1-i]

```
int arr[5] = {1, 2, 3, 4, 5};
int newArr[size];
for(int i=0 ; i<size ; i++){
    newArr[i] = arr[size-1-i];
}
```

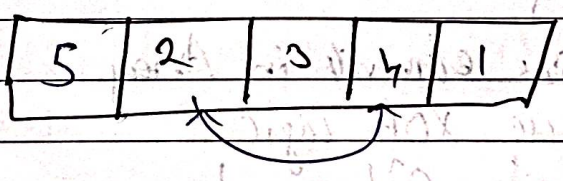
reversing array

```
for(int i=0 ; i<size ; i++){
    cout << newArr[i] << " ";
}
```

Reverse an Array using Swapping

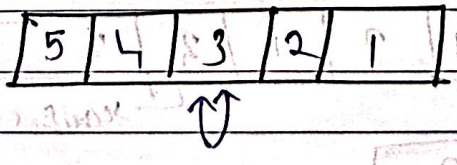


start = 0
end = 4 & swap



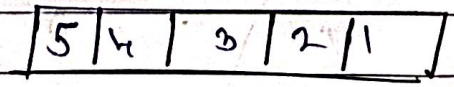
start ++
end --
& swap

start = 1
end = 3



start = 2
end = 2

start ++
end --
& swap



```
void reverse (int arr[] , int n){
    int start=0;
    int end = n-1;

    while (start <= end){
        swap( arr[start], arr[end]);
        start ++ ;
        end -- ;
    }
```

Alternate swap

```
int alt_reverse (int arr[], int n) {
    for(int start=0, end= start +1 ; start <= n-1; start += 2
                                                  end = start +1 )
    {
        if ( start+1 < n){
            swap(arr[start], arr[end]);
        }
    }
```

To find unique elements in Array

⇒ use XOR logic

$0 \wedge 1 = 1$

$0 \wedge 0 = 0$

→ | 1 | 2 | 1 | 3 | 2 |

→ unique is 3

| 2 | 3 | 2 |

$2 \wedge 3 \wedge 2$

8 4 2 1

$1\ 0\ 2$

$1\ 1\ 3$

$0\ 1$

$0\ 1$

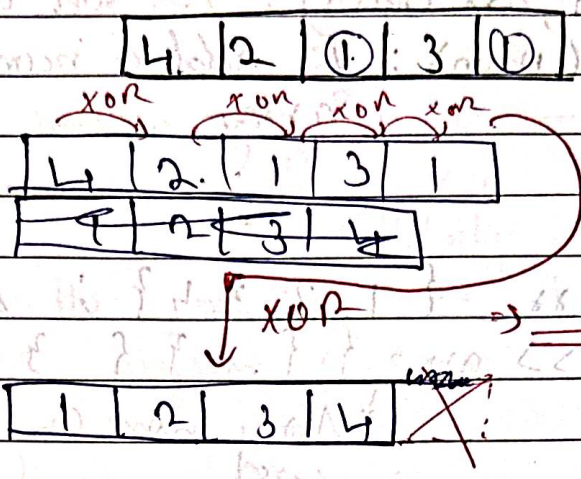
$1\ 0$

$1\ 1 = (3)$

__/__/____

Finding duplicate

n = 5



1 is the duplicate

first we need to XOR all elements - 4^2^1^3^1 & we will get an (ans)

we need to do XOR → ans ^ 1 ^ 2 ^ ... ^ n-1 = 1

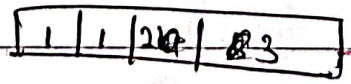
```
int findduplicate(int arr){
    int ans=0
    for(int i=0; i<arr.size(); i++){
        ans = ans^arr[i];
    }

    for(int i=1; i<arr.size; i++){
        ans = ans^i;
    }
    return ans;
}
```

XORing elements
4^2^1^3^1
5
5-1

ans=5
4
5^1^2^3^4

ans = 4
ans^1 =
4^1^2^3^4

Easy method



n = 4

expected sum = $\frac{n*(n-1)}{2} = \frac{4*3}{2} = 6$

sum = 1+1+2+3 = 7

7-6 = 1

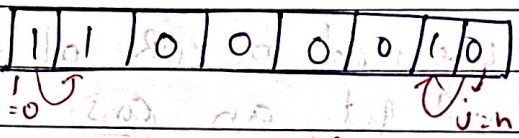
↳ use only when the array is has 1, 2, 3.. in order not like 1, 1, 4, 19

__/__/____

vector<int> arr ; To initialize an empty dynamic Array
sort(arr.begin(), arr.end()); To sort array elements.
arr.push_back(num); To add a number to a dynamic array.

vector<int> arr = {1, 2, 3, 4}; // Dynamic Array
vector<vector<int>> arr = {{ }, { }, {2}}
↳ Array inside an Array.

Sort 0s!




```
int arr[8] = {1,1,0,0,0,0,1,0};

void sort (int arr[], n){
    int i=0, j=n-1;
    while (i<j){
        if (arr[i]==0 && i<j){
            i++;
        }
        if (arr[j]==1 && i<j){
            j--;
        }
        if (arr[i]==1 && arr[j]==0 && i<j){
            swap(arr[i], arr[j]);
            i++;
            j--;
        }
    }
```

Lecture-11

Time & space Complexity.

__/__/____

Time Complexity : Time taken by the algorith to run.
→ as a function of length of ip input.

Big O notation → Theta Θ → Omega Ω
upper Bound for avg-case Complexity — Louber bound

→ shows the worst case ie it shows the max time the program can run

→ Constant time → O(1) eg → for(i=0 → 10) { cout << "Hello"; } — 10 is Constant

→ linear time → O(n) eg → for(i=0 → n) { } — n is given by input

→ Logarithmic time → O(logn) → Binary Search
→ Quadratic time → O(n²) → two for loops
→ Cubic time → O(n³) → three for loops

O(n!)
O(2ⁿ)
O(n³)
O(n²)
O(nlogn)
O(n)
O(logn)
O(1)

↑ Complexity

Time Complexity

Constraints	
$\leq [10 - 11]$	$O(n!)$; $O(n^6)$
$< [15 - 18]$	$O(2^n * n^2)$
< 100	$O(n^4)$
< 400	$O(n^3)$
< 2000	$O(n^2 * \log n)$
$< 10^4$	$O(n^2)$
$< 10^6$	$O(n \log n)$
$< 10^8$	$O(n)$, $O(\log n)$

eg $1 < n < 10^6$
$1 < n < 1000$

If constraint is $< 10^8$ use ↑ or if we use any other than $O(n)$, $O(\log n)$ it will show TLE

Space Complexity: memory taken by the algorith to run as a function of length of input.

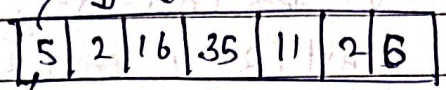
```
int a;  →  O(1)
int b;
```

Lecture-12

Binary Search

__/__/__

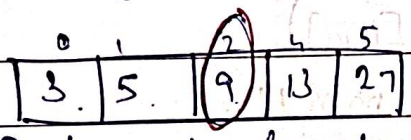
Linear search




```
for (          )
{  if(arr[i] == key)
     return i (position)
}
return -1
```

Binary search :

Can be used in Sorted Array



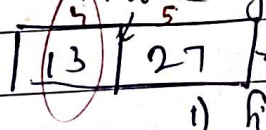
Key = 13

1) find mid element
2) Compare if it is equal to key (13)
3) If not check Key > mid or Key < 9

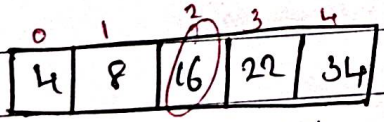
if Key > mid

13 > 9

move right 4+5 = 4

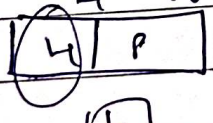


1) find mid element & check if it is equal
if equal return mid.



Key = 4

4 < 16



4

__/__/____

```
int BinarySearch (int arr[], int n, int k) {
    int start = 0, end = n-1;
    int mid = start/2 + end/2 ;
}
```

key

$\frac{end - start + start}{2}$

```
    while (start <= end) {
        if (key == arr[mid]) {
            return mid;
        }
        else if (key > arr[mid])
        {
            start = mid+1;
            end = n-1;
        }
        else {
            start = 0;
            end = mid-1;
        }
    }
    return -1;
}
```

Strings

__/__/____

Step 1.3

C++ STL (Standard Template Library)

C++ STL → 1. Algorithms
2. Containers
3. Functions
4. Iterators

Pairs

```
pair<int, int> p = {1,3}
cout << p.first << " " << p.second;

pair<int, pair<int, int>> p = {1, {3, 4}};
cout << p.first << " " << p.second.first << " " << p.second.second;

pair<int, int> arr[] = {{1,2}, {4,3}, {4,1}, {5,7}}
cout << arr[i].second;
```

✓ vectors

```
vector<int> arr;          arr = { }
arr.push_back(5);         arr = {5}
arr.emplace_back(6);      arr = {5,6}
```

↳ faster than push_back

```
vector<pair<int,int>> arr;
arr.push_back({1,2});
arr.emplace_back(1,2);

vector vector<int> arr(5,100);
{100, 100, 100, 100, 100}

vector<int> v(5) = { , , , , }
```

5 Copy one vector to another →

__/__/____

To copy an Array

```
vector<int> v1(5, 20);   // {20, 20, 20, 20, 20}
vector<int> v2(v1);      // "
```

v.clear();
↳ will clear the array.

v.empty();
↳ If array has no element → will return True else False

remove / erase an element from array

```
v = { 10, 20, 12, 14 };
v.erase( v.begin() + 1 );
```

v.begin() v.begin()+1

v = { 10, 20, 12, 14 };

v.begin()+2 v.begin()+3

```
v.pop_back();   -> {10, 20, 12}
```

Remove elements from one range to another

e.g. { 10, 20, 30, 40, 50, 60 }

to remove from 20 - 40

```
v.erase( start, end );
v.erase( v.begin()+1, v.begin()+4 );
```

To Insert elements in Array

```
vector<int> v(2, 100);                    // {100, 100}
v.insert( v.begin(), 300);                // {300, 100, 100}
v.insert( v.begin()+1, 2, 10);            // {300, 10, 10, 100, 100}

vector<int> arr(2, 50);                   // {50, 50}
v.insert( v.begin(), arr.begin(), arr.end());
                                          // {50, 50, 300, 10, 10, 100, 100}
```

List is Similar to Array

```
list<int> ls;
ls.push_back(2);          {2}
ls.emplace_back(4);       {2,4}
ls.push_front(5);         {5,2,4}
ls.emplace_front();       {2,4}
```

Stack LIFO

```
stack<int> st;
st.push(1);               {1}
st.push(2);               {2,1}
st.push(3);               {3,2,1}

cout << st.top();         = 3
st.pop();                 {2,1}
```

Queue FIFO

```
queue<int> q;
q.push(1);                {1}
q.push(3);                {1,3}
q.push(5);                {1,3,5}
```

q.back()+=5 => 5+5 = {1,3,10}

```
cout<<q.front();          1
```

q = {1,3,10}

```
q.pop();                  {3,10}
```

push(), pop() → log(n)
top() → O(1)

__/__/__

Priority_Queue

↳ Sorted in Descending order

max heap

```
priority_queue<int> pq;
pq.push(5);        {5}
pq.push(2);        {5, 2}
pq.push(8);        {8, 5, 2}
pq.push(10);       {10, 8, 5, 2}

cout << pq.top();  10
pq.pop();          {8, 5, 2}
```

Priority_Queue

↳ to sort in Ascending order [Sorting]

min Heap

```
priority_queue< int, vector<int>, greater<int> > pq;

pq.push(5);        {5}
pq.push(2);        {2, 5}
pq.push(8);        {2, 5, 8}
```

Sets

__/__/____

Sets

→ Sorted & Unique

```
set<int> st;
st.insert(1);        {1}
st.emplace(2);       {1, 2}
st.insert(3);        {1, 2, 3}
st.insert(3);        {1, 2, 3}
st.insert(4);        {1, 2, 3, 4}
```

st.erase(4) → {1, 2, 3}

Multiset → Only Sorted

ms.insert(1) → {1}
" → {1, 1}
" → {1, 1, 1}

ms.erase(1) = { }

ms.erase(ms.find(1)); → {1, 1}

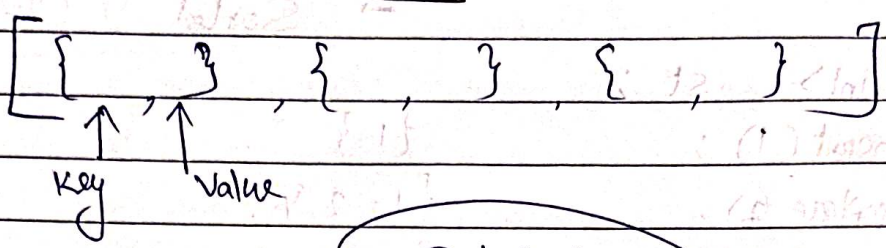
unordered set → Only UNIQUE

unordered_set<int> st;

Used to get out duplicate elements.

__/__/____

Map



Sorted Keys

```
map < int , int > mpp ;
      ↑key   ↑value

map < int , pair < int, int > > mpp ;
      key      value
```

```
mpp[1] = 2                          [{1,2}]
mpp.emplace({3,1});                 [{1,2} {3,1}]
mpp.insert({2,4});                  [{1,2} {2,4} {3,1}]
mpp.({2,3}) = 10;                   [{1,2} {2,4} {3,1} {{2,3},10}];

cout << mpp[1];  = 1
cout << mpp[5];  = 0
```

Multimap → duplicate values can be Stored
Unordered map → not Sorted

__/__/____

Algorithms

Sorting

~~a: {3, 4, 1, 6, 7}~~

a: {4, 1, 3, 2, 5}

```
sort (a, a+n);
sort (a.begin() , a.end());
```

To sort elements in particulas range

```
sort (a+2, a+4);
```

~~{4, 1, 3, 2, 5}~~ {4, 1, 2, 3, 5}

Reverse sorting

```
sort (a, a+n, greater<int>);
sort (a.begin() , a.end() , greater<int>);
```

1:0:00hr ∈ **Sorting a pair**

```
pair< int, int> a[] = { {1,2}, {2,1}, {4,1} };
```

→ Sort it acc to second element
→ If second element is same. then sort it acc to first element in desc order

=> sort (a, a+n, comp)

```
bool comp (pair<int, int> p1, pair<int,int> p2) {
    if (p1.second < p2.second) return true;
    if (p1.second > p2.second return false;
    // if they are same
    if (p1.first > p2.first) return true;
    else return false;
```

1 = Set bit

__/__/____

Binary 7 = 111

```
int num = 7
int bin = __builtin_popcount ();
          ↓will return no. of is (set bit)
          bin = 3
num = 6  → 110
          bin = 2
```

```
long long num = 163414389175633;
int bin = __builtin_popcountll();
```

To find all the permutations in an string

```
string s = "213";
sort (s.begin(), s.end());
do {
    cout << s << endl;
} while (next_permutation (s.begin(), s.end()));
```

To find max element in an Array

```
int max = *max_element (a, a+n);
```

__/__/____

To count the digit → cnt = (int) (log10(n) + 1) ;

To check Prime no. eg → 7

```
int n;
cin>>n;
int cnt=0;
for(int i=0; i*i<=n ; i++){
        if (n%i==0){
            cnt++;
            if ((n/i)!=i) {
                cnt++;
            }
        }
}
if (cnt==2) cout << "true";
else cout << "false";
```

i=28 1,2,7 14 28 — so 0,1,2

checking for 1

checking for 1

only divisible by 1 & the no.

Note: → At the end of Program use return 0; to avoid TLE

__/__/____

1.5 **Recursion** → when a fxn calls itself until a specified condition is met

```
void fxn()
{
    cout << "1";
    fxn();
}
int main()
{
    fxn();
}
```

Stack overflow → when there is numerous fxn calls waiting due to recursion.

This will go on & on

This will be waiting to complete.

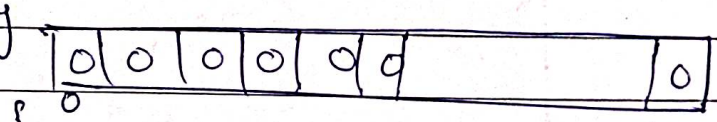
Hashing

used to count the numbers

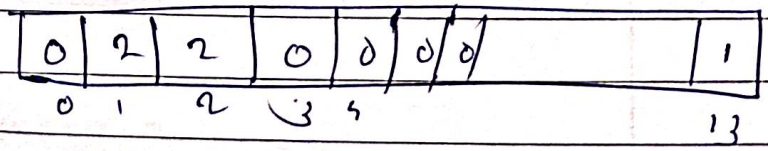
arr[5] = {1, 2, 13, 1, 2}

→ to count the number of times the number appears we make a hash array

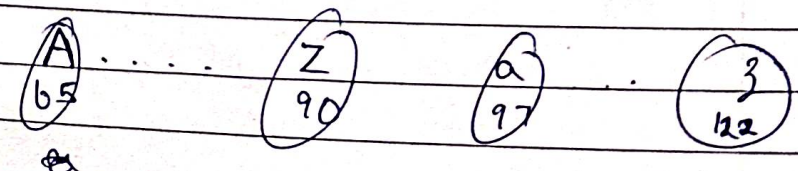
hash[13+1] = {0}




```
for(          ){
    hash[arr[i]] += 1;
}
```



ASCII value



__/__/____

position of the ~~element~~ character = char - 'a'

eg d → d - a
position of d = 100 - 97 = 3 : d

```
S = "abcmnabd";
hash[26] = {0};
for(        ){
    hash[ S[i] - 'a' ] ++;
}
```

c is the char we need to count

```
cout << hash[c - 'a'];
```

use hash[256] so that don't need to do ~~hash~~ hash[S[i] - 'a']

MAP → (key → most freq element)
select code.

unordered map : | 1 | 2 | 3 | 1 | 3 | 2 | 12 |

key → value → how many times it appears

map < int , int >

map < 1 , 2 >

mpp[1] = 2

(12→1) (3→2) (2→1) (1→2)	→	(12→1) (3→2) (2→2) (1→2)

```
int n;
cin>>n;
int arr[n];
for (int i=0 ; i<n ; i++) {
        cin >> arr[i];
    }

map<int, int> mpp;
for (int i=0 ; i<n; i++){
        mpp[arr[i]]++;
    }

for(                    ) {
    cout << mpp[i];
}

int q;
cin>>q;
while (q--){
        int num;
        cin num;
        cout << mpp[num];
}
```

arr = { 1, 2, 3, 1, 3, 2, 12 }

q = 5

num = 1
2
3
5
12

if num=1, it will print 2

mpp[1] = 2
mpp[2] = 2
mpp[3] = 2
mpp[4] = 0
mpp[12] = 1

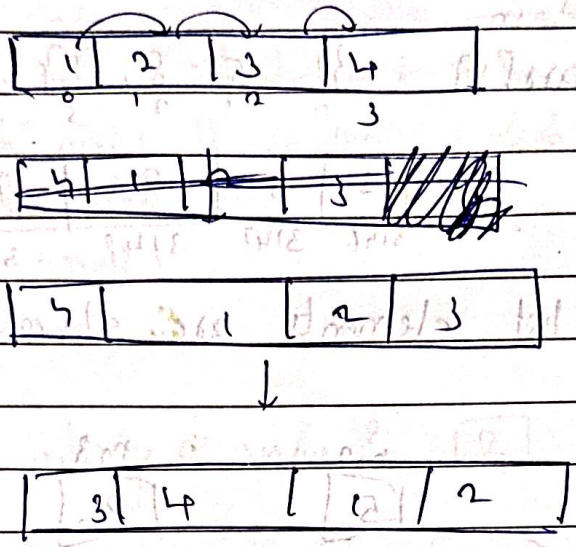
for string mapping use → map<char, int>

Hashing → Division method

__/__/____

Rotate an Array

k=2



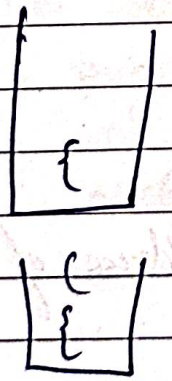
ans[(i+k)%n] = arr[i] Rotate Right

ans[(i-k+n)%n] = arr[i] Rotate left

Paranthesis Problem

{()}[] → true

)[] → false



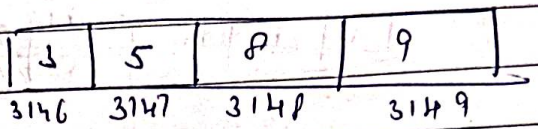
if the bracket is open push it to stack
if the bracket is close, check the top of stack, if they are of same pair
eg ( ) or { } or [ ], remove the open bracket from the stack.
Finally → if the stack will be empty, so return true.

→ used in stack & Queue

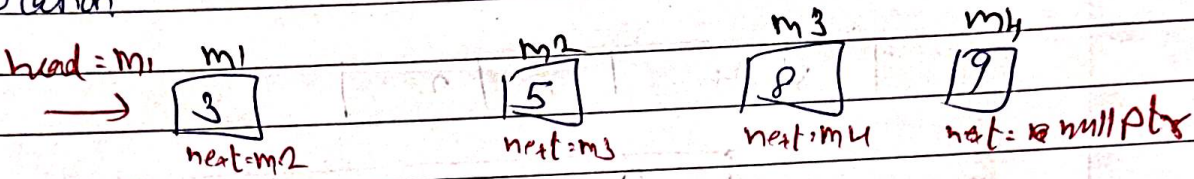
Linked List

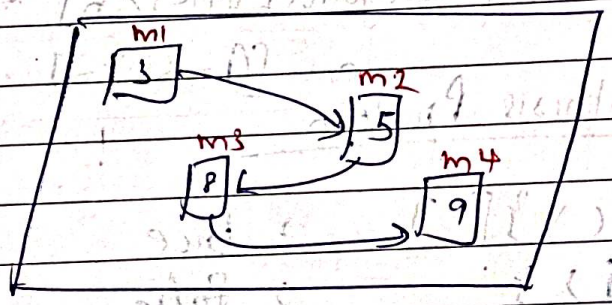
~~• elements stored in~~

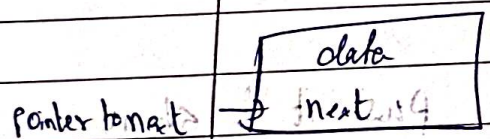
For Array arr[4] = {3, 5, 8, 9};
0 1 2 3



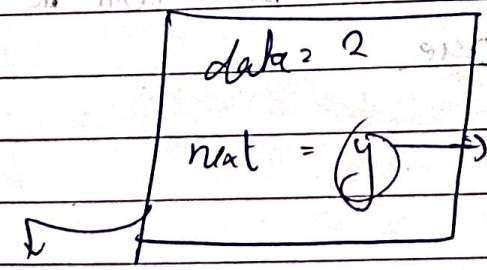
But in Linked list elements are stored in contigious location







int x = 2
int* y = &x;
cout << y; → will return the memory address of x

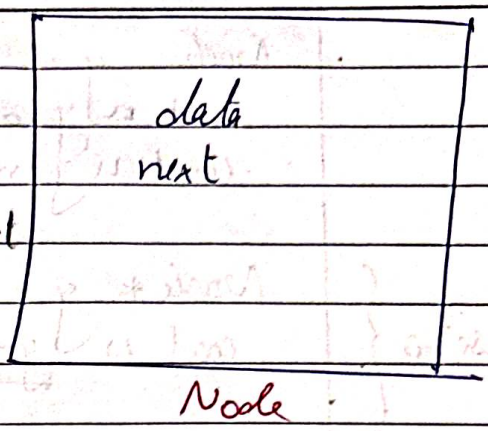


to store this we need to create a data type → struct node

__/__/____

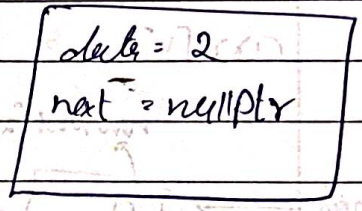
class

struct Node {
  int data;
  Node* next; → to store pointer to the next
Node(data1, next1){ to initialize data & next
  data = data1;
  next = next1;
}



How to initialize a value? 2

{ Node x = node(2, nullptr);
{ Node *y = &x; → to get pointer



or

to get pointer → Node* y = new Node(2, nullptr);

Code → #include <bits/stdc++.h>
using namespace std;
class Node {
  public:
  int data;
  Node* next;

  public:
  Node(int data1, Node* next1){
    data = data1;
    next = next1;
  }
};
int main(){
  vector<int> arr = {2, 5, 8, 7};
  Node y = Node(arr[0], nullptr);
  cout << y; → return 2

Node* y = new Node(arr[0], nullptr);
cout << y; → return address

__/__/____

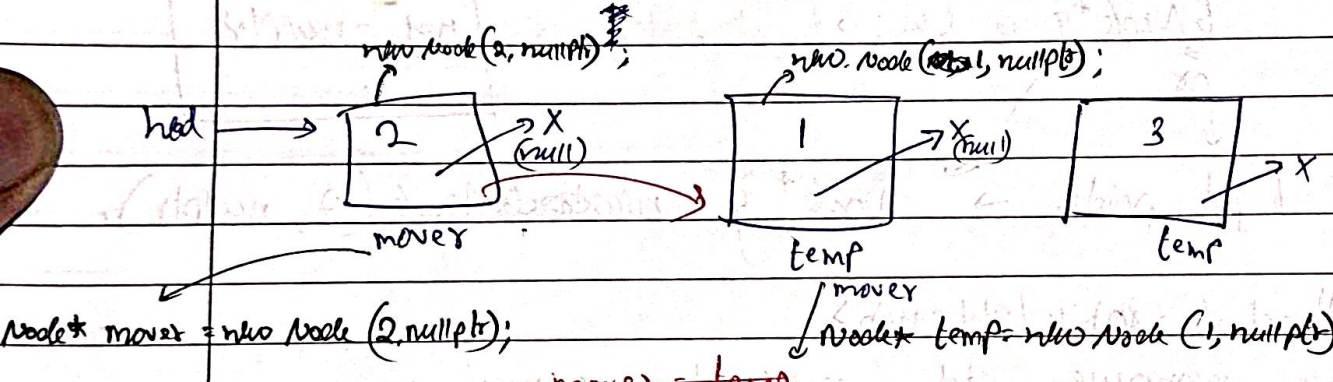
```
Node y = Node(arr[0], nullptr);
cout << y.data;   -> returns 2
cout << y.next;   -> returns 0x0 -> (nullptr)
```

use this

```
Node* y = new Node(arr[0], nullptr);
cout << y->data;   -> will return 2
```

~~Convert Array → Linked list~~

arr[4] : [2, 1, 3, 8]



~~mover = temp~~
mover = next = temp

Convert Array to Linked List we should return only the head

```
Node* ConvertArr2LL(vector<int> &arr) {
    Node* head = new Node(arr[0]);
    Node* mover = head;
    for (int i=0; i<4; i++) {
        Node* temp = new Node(arr[i]);
        mover->next = temp;
        mover = temp;
    }
    return head;
int main() {
    vector<int> arr = {2, 3, 5, 7};
    Node* head = Convert Arr 2 LL (arr);
```

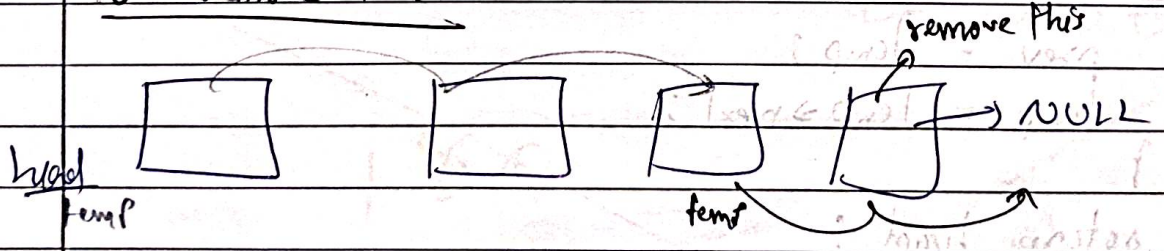
Traversal

```
int main(){
    vector<int> arr = {2,5,8,7};
    Node* head = ConvertArr2LL(arr);
    Node* temp = head;
    while(temp){
        cout << temp->data << " ";
        temp = temp->next;
    }
}
```

To remove head

```
Node* removesHead(Node* head){
    if(head==NULL) return head;
    Node* temp = head;
    head = head->next;
    delete temp;
    return head;
}
```

To remove tail

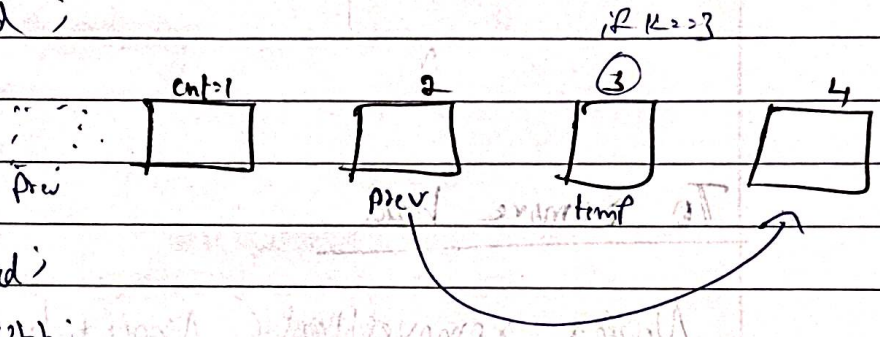



```
Node* removeTail(Node* head){
    if(head==NULL || head->next==NULL) return NULL;
    Node* temp = head;
    while(temp->next->next != NULL){
        temp = temp->next;
    }
    delete temp->next;
    temp->next = nullptr;
    return head;
```

__/__/____

Delete K^{th} element

```
Node* removeK (Node* head, int k) {
  if ( head == NULL) return head ;
  if (k == 1) {
        Node* temp = head;
        head = head->next;
        delete temp ;        or  free(temp)
        return head ;
  }
```

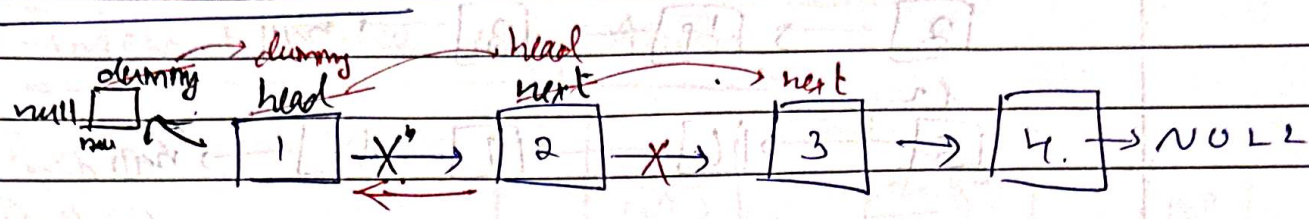



```
int cnt = 0;
Node* temp = head;
Node* prev = NULL;
while ( temp != NULL) {
      cnt ++ ;
     if (cnt == k) {
          prev -> next = prev -> next -> next;
          free (temp);
          break;
      }
    prev = temp;
    temp = temp -> next;
}
return head;
```

__/__/____

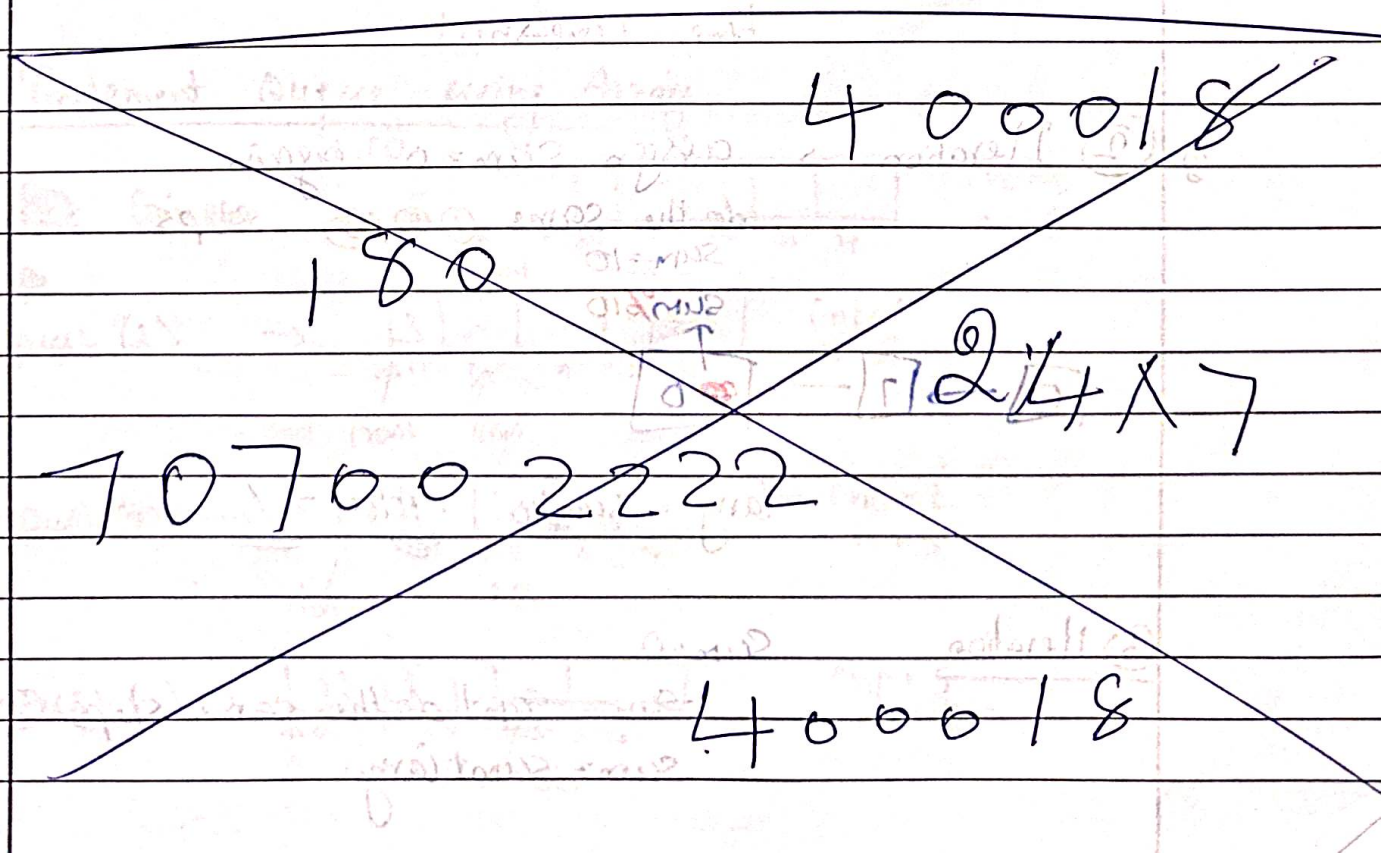
SDE Sheet

Reverse the LL

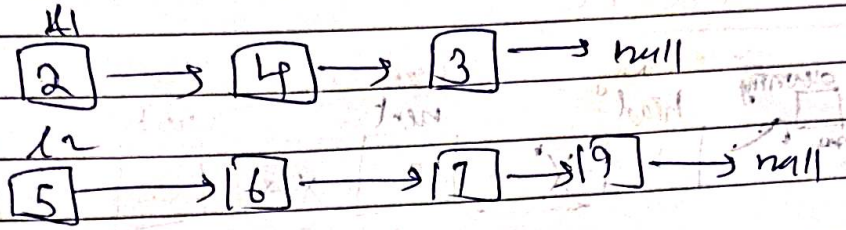


1. Create a dummy node NULL before 1
2. 1 is head
3. for first iteration → create next node dummy
4. Break 1 → 2 & point 1 to null

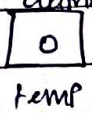
dummy = head
head = next
next → 3



2. Add Two Numbers



1) Create a dummy node



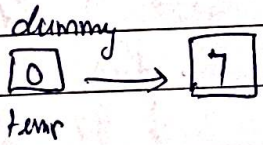
sum = 0

2) ① Iteration → check if l1 is != NULL

① sum = sum + l1->val ; // 2

② check if l2 != NULL

③ sum = sum + l2->val // 5



l1 = l1->next

l2 = l2->next

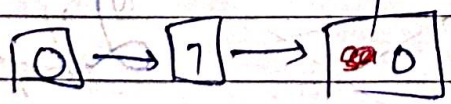
temp = temp->next

3) ② Iteration → assign sum = 0 again

do the same ①②③ steps

sum = 10

sum%10



carry = sum/10 = 10/10 = 1

④ Iteration sum = 0

~~sum = sum~~ do the same steps ①②③

sum = sum + carry

__/__/____

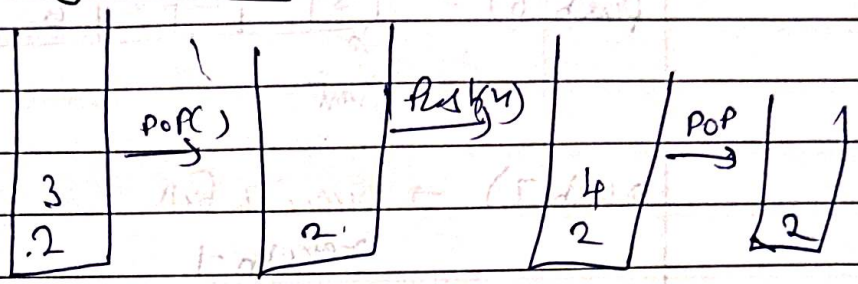
DAY-13

Stack & Queues

Implement stack using Arrays

LIFO

Push (2)
Push (3)
pop ( )
push (4)
pop ( )



OR

To return the popped element

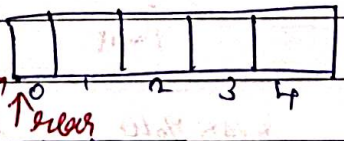
~~to return the~~

```
void Push (int x) {
        arr[++top] = x;
    }

void pop ( ){
    if (top >= 0){
        return arr[top--];
    } else{
        return -1;
    }
```

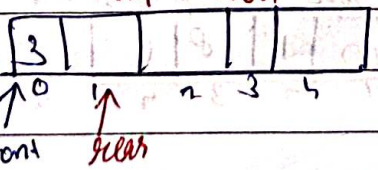
Implement Queue using Array

FIFO

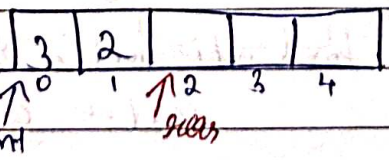
Given n = 5



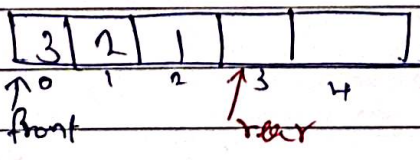
push (3) →

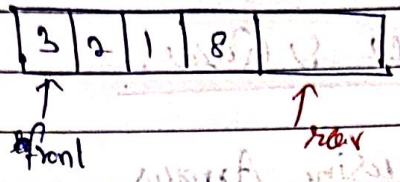


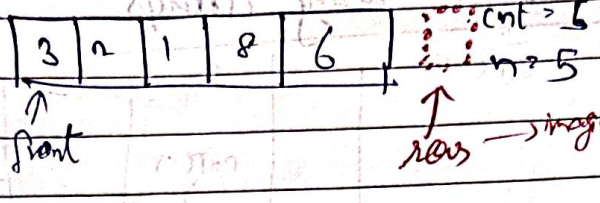
push (2) →



push (1) →

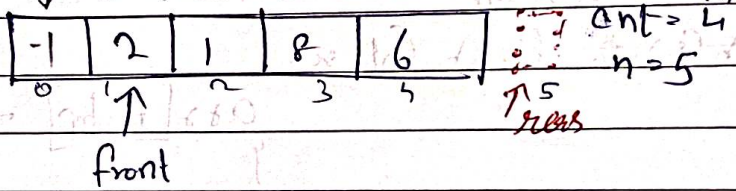


push(8) → 

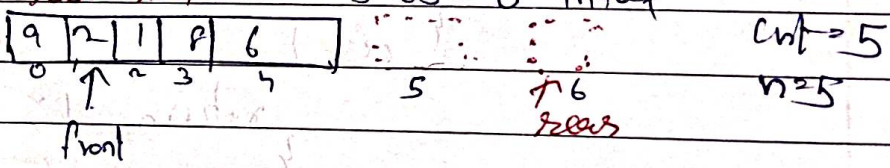
push(6) = 

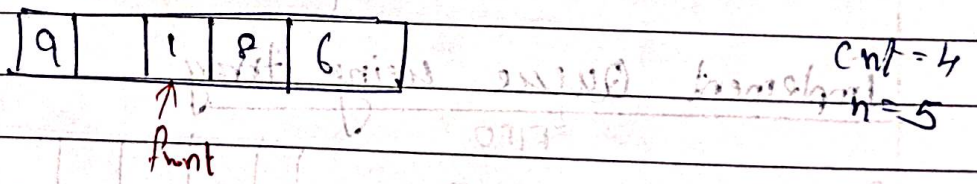
push(7) → Queue is full
return -1

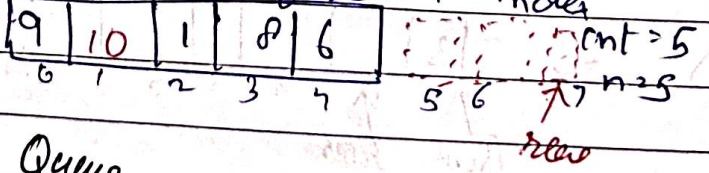
top() → a[front]
a[front % n] → a[0 % 5] = 0

pop() → 

top() → a[front] = 2

→ push(9) → rear % n = 5 % 5 = 0th index 

pop() → 

push(10) → rear % 10 → 6 % 5 = 1st index 

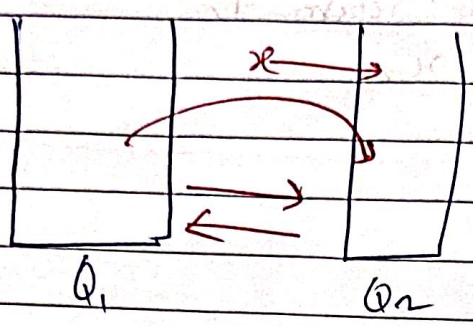
<u>To print the Queue</u>

```
for(int i = front; rear-1 ; i++){
    cout << a[i % n] ;
```

Code:

```
Push(x) {
        if (cnt == n)  return -1;
        a[ rear %n] = x;
        rear ++)
        cnt ++;
    }
pop(x) {
        if (cnt == 0)

        a [ front %n] = -1;
         front ++;
         cnt - -;
    }
top( ) {
        if (cnt == 0) return -1;

        return  a[ front % n];
    }
```

225 Implement stack using Queue



Steps

Push(x)

→ 1) Add x → Q2

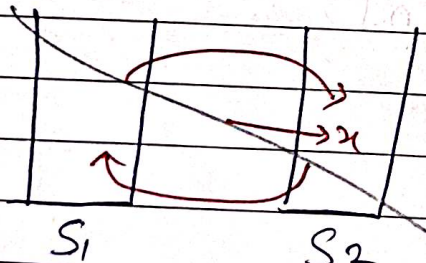
2) Q1 → Q2 (Element by element)

3) Swap Q1 ⇌ Q2

pop() {

Remove the top of Q1

Implement Queue using stack



Push(x)

1) S1 → S2

2) x → S2

→ S2 → S1

pop(){

S1.pop()

}

Easy method

__/__/__

```
top() {
  if (output not empty) {
     return output.top();
  }
  else {
     input → output
     output.top();
  }
```

```
Push (x) {
   add x → input
}

pop() {
   if (output not empty) {
      output.pop()
   }
   else {
      input → output
      output.pop()
   }
```

L7 Lp. Love Babbar

__/__/____

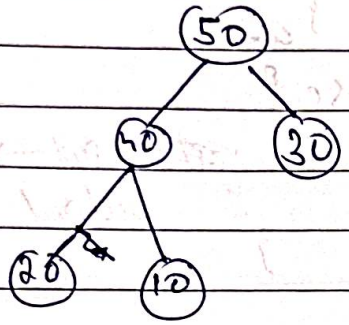
HEAPS

is a complete binary tree CBT that comes with heap order property

CBT → every level is completely filled except the last
→ nodes always added from left

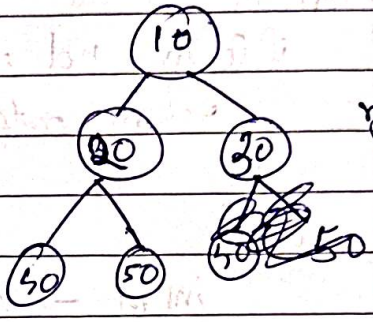
Heap Order Property

Max heap



left > right

min heap

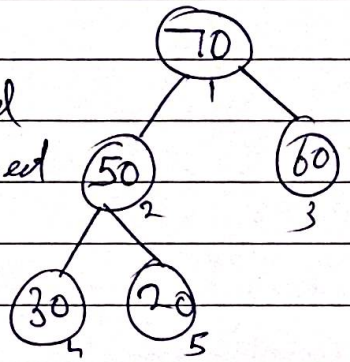


right > left

Max Heap

Insertion

1) Insert at the end
2) Taking it to correct position by comparing value with parent



Node = ith index
left child = 2*i index
Right child = 2*i + 1 index
Parent = i/2

0	70	50	60	30	20	
0	1	2	3	4	5	6

```
void insert (int val){
    size = size+1;
    int index = size;
    arr[index] = val;
    // checking
    while (index > 1){
        int parent = index/2;
        if (arr[parent] < arr[index]){
            swap (arr[parent], arr[index]);
            index = parent;
        }else return;
```

```
heap(){
    arr[0] = -1;
    size = 0;
}
```

__/__/____

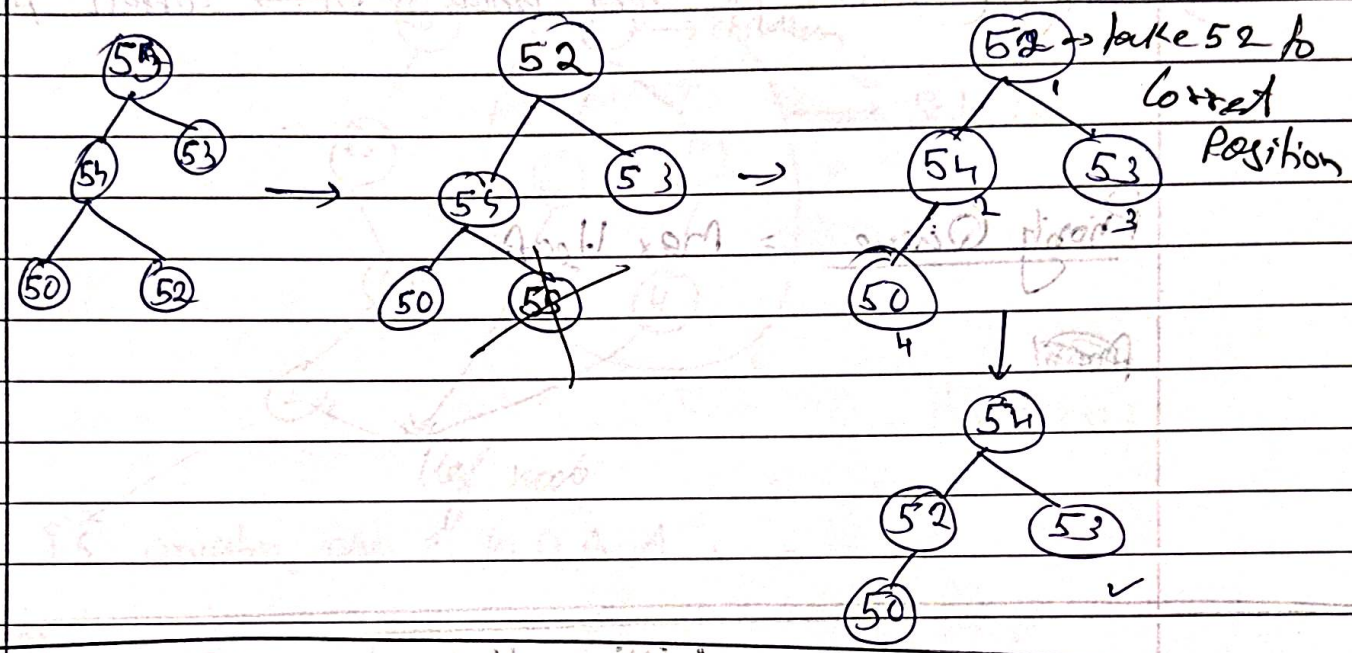
Deletion → of Root node

Step 1) → Swap root node with last node

arr[i] = arr[n]

Step 2) → delete the last node

Step 3) → Take the Root node to correct position

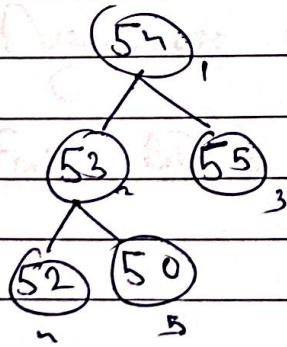


Heapify Algorithm

Build Max Heap

i/p → | x | 54 | 53 | 55 | 52 | 50 |

0 1 2 3 4 5



CBT → leaf node is from (n/2 + 1) → n

n = 5

(5/2 + 1) → 5

3 → 5 are leaf node

we need to check from right to left from n/2 → 1

```
for( i = n/2 → 1){
        heapify (arr, size, i);
}
void heapify (int arr[], int n, int i){

}
```

__/__/___

Heap Sort

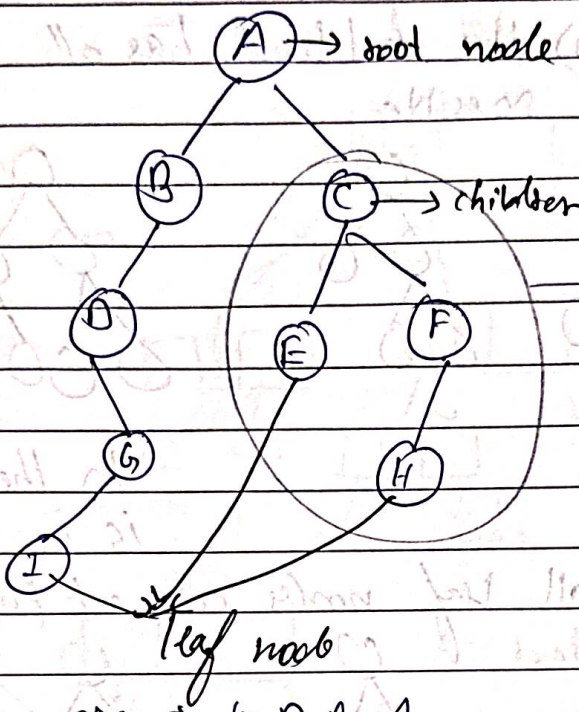
step 1) Swap the last node with first node
2) size --
3) Heapify (Place the root node ~~to the~~ in the correct position)

Priority Queue = Max Heap

[redacted]

L 75 Heaps Question Solving — Love Babbar

Binary Tree

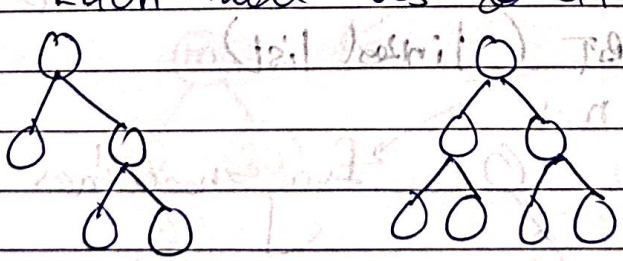


I's ancestors are → G, D, B, A . . .

There are 5 types of BT

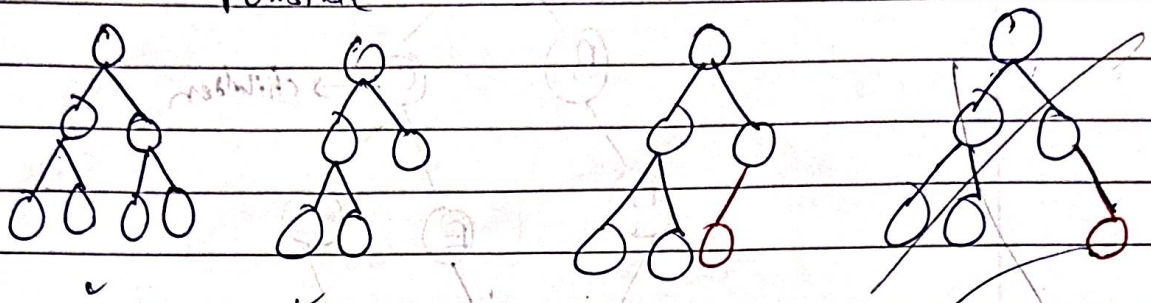
1) Full Binary Tree
2) Complete Binary Tree
3) Perfect BT
4) Balanced BT
5) Degenerate BT

1. Full BT → ~~children has either 0 or 2 nodes~~
Each node has either 0 or 2 nodes



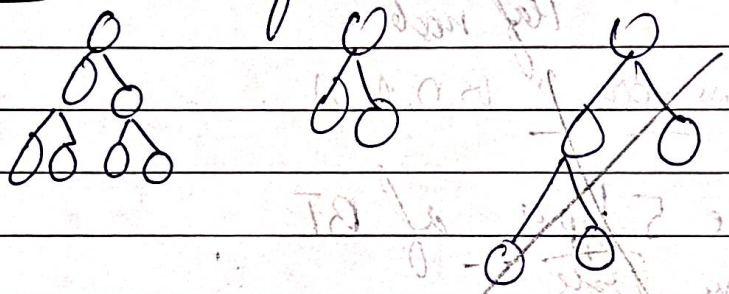
This node has only one children

2) Complete BT → 1) All levels are completely filled except the last level
2) the last level has all nodes in left as possible

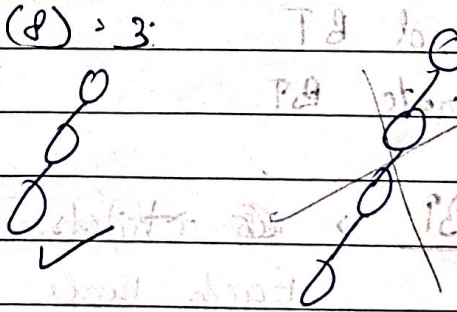


In the last level the node is in Right. that not possible

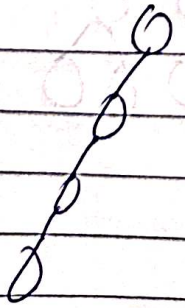
3) Perfect BT - all leaf nodes are at the same level



4) Balanced BT - height of tree is max of log(n) nodes
ex for eg n=8
$\log_2(8) = 3$

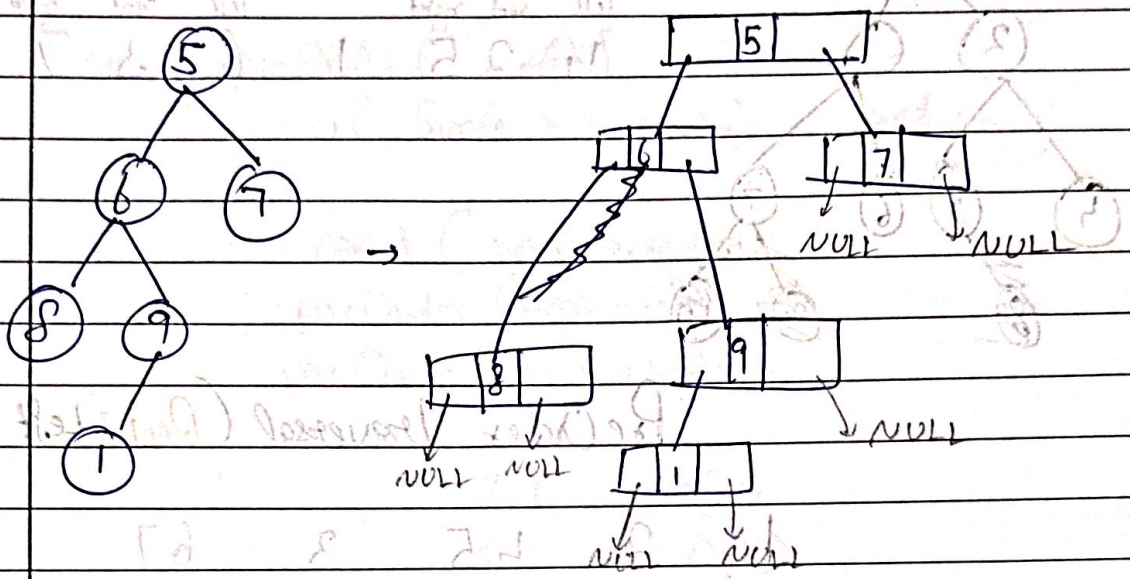


5) Degenerate BT (Linked list)
n=4 → Every node has single children



__/__/____

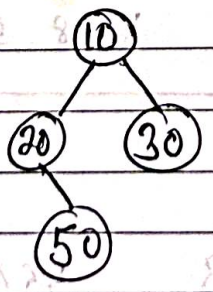
BT representation in C++



← root node.

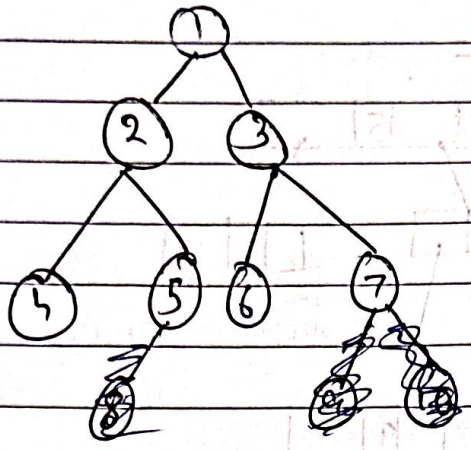
```
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
    Node(int val){
        data = val;
        left = right = NULL;
    }
};
```

```
int main() {
    struct Node* root = new Node(10);
    root->left = new Node(20);
    root->right = new Node(30);
    root->left->right = new Node(40);
```



Traversal techniques (DFS)

__/__/__



In Order Traversal (Left Root Right)

left root right left root right

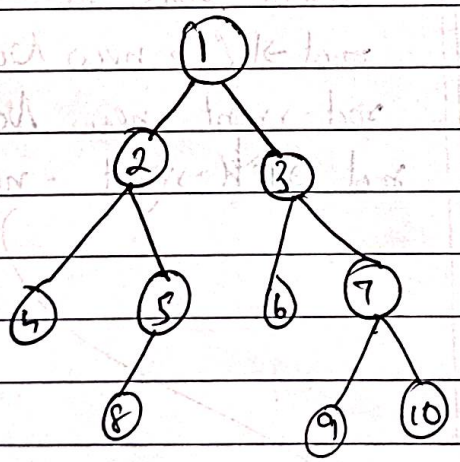
4 2 5 1 6 3 7

Pre Order Traversal (Root Left Right)

1 2 4 5 3 6 7

Post Order Traversal (Left Right Root)

4 5 2 6 7 3 1



In order Traversal

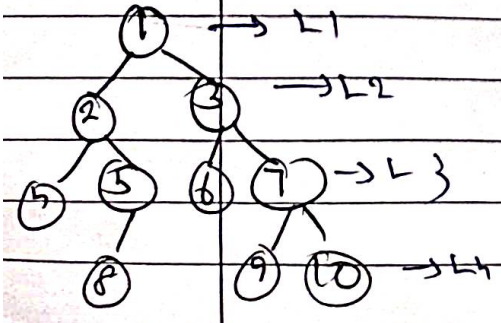
4 2 8 5 1 6 3 9 7 10

Pre Order Traversal

↳ 1 2 4 5 8 3 6 7 9 10

Post → 4 8 5 2 6 9 10 7 3 1

BFS



1 2 3 4 5 6 7 8 9 10

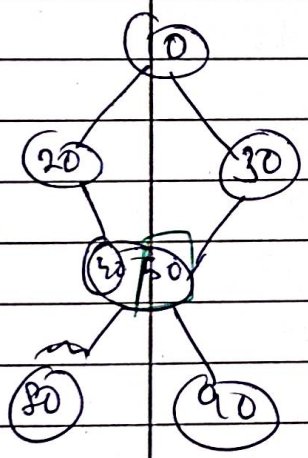
__/__/____

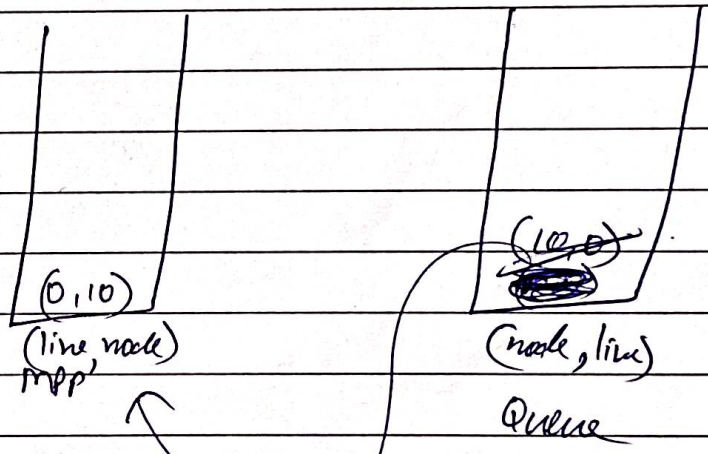
C++ code for Pre Order → (Root → left → Right)

```
void preOrder ( node) {
    if (node == NULL) return ;

    print ( node → data) ;
    preOrder (node → left) ;
    preOrder (node → right) ;
}
```

TC = O(N)
SC → O(N)

Bottom View of Tree





10, 20, 80, 50, 90, 30

node = 10
line = 0